UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ingeniería

Previo

Documentación

El proceso de arranque de una computadora

Materia:

Sistemas Operativos

Integrantes:

González Martínez Fernando

Quezada Olivares Emir

Profesor:

Gunnar Eyal Wolf Iszaevich

Grupo 7 | Semestre 2026-2

Índice

Introducción..... 3
Suministro de corriente..... 3
Carga de la BIOS/UEFI..... 4
Almacenamiento y particiones MBR/GPT..... 6
Carga del boot manager..... 7
Gestor de arranque (OS Loader)..... 8
Carga del sistema operativo..... 9
Conclusión..... 10
Bibliografía..... 11

Introducción

El proceso de arranque de una computadora muchas veces pasa desapercibido para los usuarios; sin embargo desde el momento en que se presiona el botón de encendido, ocurren múltiples eventos coordinados entre el hardware y el software que permiten preparar el equipo para su uso.

En este documento se analizaran cada una de las etapas del arranque, desde la inicialización del hardware hasta llegar a la carga del sistema operativo, describiendo su propósito, funcionamiento y relevancia dentro del sistema.

Suministro de corriente

La Fuente de Alimentación (PSU) es el componente encargado de convertir la corriente alterna (AC) proveniente del enchufe de la pared en corriente continua (DC) de bajo voltaje, la cual es utilizable por los componentes internos del ordenador. No es solo un adaptador, sino un regulador que garantiza la estabilidad del sistema.

Cuando se presiona el botón de encendido en la PC, se cierra un circuito eléctrico que envía una señal a la PSU (Power Supply Unit, por su abreviatura en inglés) para que inicie su operación. Antes de entregar energía total al sistema, la fuente realiza un Self-Test o autodiagnóstico en español:

- La fuente de alimentación verifica que las líneas de voltaje (+3.3V, +5V y +12V) sean estables.
- Si el autodiagnóstico es exitoso, la fuente de alimentación envía una señal digital al procesador (PWR_OK - Power Good) a través del conector de la tarjeta madre. Esta señal le indica a la CPU que la energía es segura y que puede iniciar el proceso de arranque (boot).
- Si la fuente detecta irregularidades, no enviará la señal que permita avanzar, lo que resultará en un sistema que no enciende o ventiladores que giran apenas un segundo antes de detenerse para proteger los componentes.

Nota técnica: Es fundamental señalar que este proceso varía en dispositivos móviles y SoCs (System on Chip), donde existe un controlador de corriente independiente del CPU que gestiona el encendido. Además, bajo el estándar ATX, una computadora moderna nunca está “totalmente” apagada; permanece en un estado de espera que permite funciones como Wake on LAN (encendido por red) y la ejecución del SMM (System Management Mode), un modo de operación de alta

prioridad del procesador que gestiona tareas de energía y seguridad de forma invisible para el sistema operativo.

La mayoría de las computadoras modernas siguen el estándar ATX (Advanced Technology eXtended), que define la disposición y tipos de conectores:

Conector ATX de 24 pines: Es el suministro principal para la placa base, alimentando el chipset, la memoria RAM y los buses de datos.

Conector EPS/CPU (4 o 8 pines): Está dedicado exclusivamente a suministrar energía al procesador, y las CPUs modernas de alto rendimiento suelen requerir hasta 8 pines (o incluso dos conectores de 8) debido a su alta demanda de corriente.

Líneas de Voltaje:

Como se mencionó anteriormente, existen diferentes líneas de voltaje, cada una con distintos objetivos:

- **+12V:** Alimenta los componentes de mayor consumo (CPU y Tarjeta Gráfica).
- **+5V y +3.3V:** Utilizados por periféricos, almacenamiento y circuitos lógicos.

Tras el lanzamiento de la señal Power Good, la lógica de control (sea un chip dedicado o el chipset integrado) libera la señal de reset del CPU, permitiendo que este comience a ejecutar sus primeras instrucciones dentro del **vector de reset**, donde posteriormente se dirigirá al firmware (BIOS) o a un paso intermedio llamado Fase SEC (UEFI)

Carga de la BIOS/UEFI

La BIOS (Basic Input/Output System) es el primer software que se ejecuta al encender una computadora. Técnicamente, se define como firmware: un tipo de software específico que proporciona control de bajo nivel para el hardware del dispositivo.

Se almacena en un circuito integrado de tipo EEPROM (Electrically Erasable Programmable Read-Only Memory). Esto permite que el BIOS sea persistente (no se borra al apagar la PC) pero actualizable mediante procesos de "flasheo". Actúa como el puente inicial entre el hardware físico y el sistema operativo (Windows, Linux, etc.). Sin el BIOS, el procesador no sabría cómo comunicarse con la memoria RAM o el disco duro.

Arquitectónicamente, el BIOS opera de forma restringida en un entorno de 16 bits (modo real), lo que limita su acceso a solo 1 MB de memoria RAM durante su ejecución.

POST (Power-On Self Test)
Una vez el procesador carga la BIOS, su primera instrucción es ejecutar el POST. Este es un inventario de seguridad que verifica los componentes esenciales:

- **CPU:** Verifica el funcionamiento básico del procesador.
- **Memoria RAM:** Se realiza un conteo y verificación de integridad.
- **Tarjeta de Video:** Inicializa el controlador gráfico para poder mostrar información en pantalla (si falla, la placa suele emitir "beeps”).
- **Periféricos de entrada:** Verifica la presencia de teclado y ratón.

Si alguno de estos componentes críticos falla o no se encuentra, la BIOS detiene el proceso de arranque para evitar daños mayores.

En la actualidad, la BIOS tradicional (Legacy) ha sido reemplazada en gran medida por **UEFI** (Unified Extensible Firmware Interface) pues tiene mayores ventajas.

La UEFI (Unified Extensible Firmware Interface) es el estándar moderno que ha reemplazado a la BIOS tradicional en los sistemas actuales desde la década de 2010. A diferencia de su predecesor, UEFI permite el uso de procesadores de 32 y 64 bits, eliminando la barrera de 1 MB de memoria y permitiendo interfaces y controladores mucho más complejos antes de que cargue el sistema operativo.

Su estructura es modular y se divide en fases estandarizadas:

1. **Fase SEC (Security):** Es la primera fase tras el vector reset. Maneja los eventos de reinicio y establece una "raíz de confianza". Debido a que la RAM aún no está disponible, utiliza el caché del CPU como memoria temporal (Cache-as-RAM).
2. **Fase PEI (Pre-EFI Initialization):** Se encarga de la inicialización mínima necesaria del procesador y el chipset para descubrir e instalar la memoria RAM permanente.
3. **Fase DXE (Driver Execution Environment):** Carga controladores para dispositivos (PCI, USB, Red). Se crea la UEFI System Table, que es la estructura de datos más importante y contiene punteros a todos los servicios disponibles.
4. **Fase BDS (Boot Device Selection):** El gestor de arranque interpreta variables almacenadas en la NVRAM (como BootOrder) para decidir qué aplicación cargar. A diferencia de la BIOS, UEFI carga archivos ejecutables completos (formato PE) desde una partición formateada en FAT32.

A diferencia de la BIOS, en la cual buscamos una firma de arranque para poder realizar la transición al esquema de almacenamiento, concluir con su ejecución y transferir el control, la UEFI sigue presentando participación activa:

5. **Fase RT (Run Time):** Mientras el sistema operativo corre, solo permanecen activos los Servicios de Tiempo de Ejecución, que permiten al SO consultar la hora, gestionar variables de NVRAM o actualizar el firmware mediante "cápsulas".

Almacenamiento y particiones MBR/GPT

El **Master Boot Record (MBR)** es el estándar de organización de sectores de arranque más antiguo. Se localiza físicamente en el primer sector de un dispositivo de almacenamiento (Sector 0).

Su función es doble, identificar dónde reside el sistema operativo para iniciar el arranque, y almacenar la "hoja de ruta" de cómo está dividido el disco (particiones).

Aunque el MBR tiene un tamaño "estricto" de 512 bytes y está diseñado para gestionar sectores lógicos de esa misma dimensión, la industria evolucionó hacia el Advanced Format (AF). Esta tecnología utiliza sectores físicos de 4096 bytes (4 KiB) la cual elimina áreas redundantes de encabezado y corrección de errores (ECC) para ganar entre un 7% y 11% de eficiencia.

Para mantener la compatibilidad con sistemas MBR limitados a 512 bytes, muchos discos operan en modo 512e (emulación), reportando sectores lógicos de 512 bytes aunque físicamente sean de 4 KiB.

Componente	Tamaño	Descripción
Código de Arranque (Boot Loader)	446 bytes	Contiene el código ejecutable primario para iniciar el SO.
Tabla de Particiones	64 bytes	Describe hasta 4 particiones primarias (16 bytes por cada una).

Firma de Unidad (Signature)	2 bytes	El "Número Mágico" **0x55AA**. Si no está, el BIOS asume que el disco no es arrancable.

Aunque a menudo se redondea el cargador de arranque a 448 bytes, en la realidad técnica son 446 bytes. Los últimos 2 bytes del sector siempre deben ser 55 AA para que el BIOS valide el sector como un MBR legítimo.

Debido a que 446 bytes son insuficientes para cargar un sistema operativo moderno (como Windows 11 o Linux), el MBR utiliza una técnica llamada **Chain Loading**:

1. El BIOS carga el código del MBR en la memoria RAM.
2. El código del MBR busca en la Tabla de Particiones cuál está marcada como "Activa".
3. El MBR entonces "pasa la batuta" (carga y ejecuta) al **VBR (Volume Boot Record)**, que es otro sector de arranque ubicado al inicio de la partición específica.

La tabla de 64 bytes permite definir cómo se divide el espacio físico. Sin embargo, el diseño original del MBR tiene una limitación crítica: **solo permite 4 particiones primarias**.

Para superar esto, se inventó la **Partición Extendida**:

- **Partición Primaria:** Divisiones directas del disco (ej. Disco C:).
- **Partición Extendida:** Una de las 4 ranuras se usa como un "contenedor" que puede albergar múltiples **Unidades Lógicas** (D:, E:, F:, etc.), permitiendo que el usuario tenga más de 4 letras de unidad.

A pesar de su eficiencia, el MBR ha sido desplazado por el estándar **GPT (GUID Partition Table)** debido a tres razones principales:

- **Límite de 2 Terabytes:** El MBR utiliza un direccionamiento de 32 bits. Matemáticamente, esto limita la capacidad máxima de un disco a $2^{32} \cdot 512$ bytes, lo que equivale exactamente a **2 TiB** (Tebibytes). Cualquier espacio por encima de eso en un disco moderno queda inutilizable.

  Esta limitación de 2 TiB es estándar para sectores de 512 bytes. Sin embargo, con el Formato Avanzado **(Advanced Format)** que utiliza sectores de 4 KB, el direccionamiento del MBR puede extenderse teóricamente hasta los **16 TB**. A pesar de esto, el primer sector de arranque se mantiene casi siempre en 512 bytes por razones de compatibilidad estricta.

- **Fragilidad:** Solo existe una copia del MBR. Si el Sector 0 se daña físicamente o por un virus, todo el disco se vuelve inaccesible (el famoso error *"Operating System not found"*).
- **Seguridad:** No posee características modernas como el *Secure Boot* (característica del firmware UEFI), que evita que rootkits se carguen antes que el sistema operativo.

Carga del boot manager

El Boot Manager es el puente entre el firmware de la máquina y el sistema operativo. Su propósito es permitir al usuario elegir qué sistema iniciar (en caso de multiboot) y preparar el entorno para el Núcleo o Kernel. La forma en que se carga varía según el esquema de almacenamiento:

1. **En sistemas MBR (Legacy BIOS):** Dentro de los sectores iniciales de la partición activa en la cual se localizó el VBR (Volume Boot Record) se encuentra el código del gestor de arranque. Este programa, al cual se le cede el control, es el que finalmente lee el sistema de archivos (que la BIOS no entiende por sí misma) para buscar y cargar el archivo del Kernel.

2. **En sistemas GPT (UEFI):** El proceso se gestiona dentro de la fase BDS (Boot Device Selection). En lugar de ejecutar código "ciego" en un sector físico, el Firmware Boot Manager de la UEFI consulta variables globales almacenadas en la NVRAM (como BootOrder y Boot####).
   Estas variables contienen una Ruta de Dispositivo (Device Path) que apunta directamente a un archivo ejecutable con extensión .EFI (un ejecutable en formato PE/COFF). Dicho archivo reside en la ESP (EFI System Partition), una partición especial formateada generalmente en FAT32 que la UEFI sí es capaz de leer de forma nativa.

En la actualidad, existe una gran variedad de gestores de arranque adaptados a diferentes arquitecturas y sistemas operativos. Algunos de los ejemplos más famosos son:

Entornos Linux

- **GRUB (GRand Unified Bootloader):** El estándar en la mayoría de las distribuciones modernas. Es un cargador de múltiples etapas que ofrece un menú de inicio avanzado con soporte para chainloading (carga en cadena). Inicializa el entorno de hardware y carga directamente la imagen del *kernel* en la memoria.

- **LILO (Linux Loader):** Un que sentó las bases del arranque en este sistema. Aunque ha sido mayormente desplazado por GRUB, sigue siendo un referente histórico importante.
- **eLILO:** La evolución de LILO diseñada específicamente para interactuar con el firmware UEFI. Suele implementarse en productos de electrónica de consumo o arquitecturas integradas que requieren compatibilidad estricta con el estándar PC.

Entornos Windows

- **Windows Boot Manager (BOOTMGR):** El gestor predeterminado en las versiones contemporáneas de Microsoft. En equipos actuales con UEFI, opera a través del ejecutable bootmgfw.efi, funcionando como una aplicación nativa que la placa base carga directamente desde la partición del sistema EFI (ESP).
- **NTLDR (New Technology Loader):** El gestor de arranque utilizado en versiones anteriores de Windows (como XP y sistemas basados en NT) antes de que Microsoft hiciera la transición hacia la arquitectura de BOOTMGR.
- **Otras Arquitecturas:**
    - **U-Boot:** El gestor de arranque estándar en sistemas embebidos y dispositivos con arquitecturas ARM o RISC-V.
    - **iBoot:** El cargador de arranque propietario de Apple, utilizado en iPhones y computadoras Mac con procesadores Apple Silicon.

Gestor de arranque (OS Loader)

Una vez que el firmware (ya sea BIOS o UEFI) localiza y ejecuta el gestor de arranque, se inicia una fase de preparación crítica antes de que el kernel tome el mando definitivo:

1. **Selección del usuario:** Si hay varios sistemas operativos instalados, el gestor de arranque presenta un menú para que el usuario elija cuál cargar. Si se presenta un único sistema operativo, el sistema lo inicializará.
2. **Identificación y carga del Kernel:** El gestor determina en qué partición y directorio reside el núcleo o Kernel del sistema operativo y lo carga en la memoria RAM.
3. **Construcción del Mapa de Memoria:** Especialmente en sistemas UEFI, el cargador debe obtener un mapa detallado de los recursos de memoria física a través de las APIs del firmware. Esto es vital para que el Kernel sepa qué áreas de la RAM están libres y cuáles están reservadas para el hardware.

4. **Configuración del entorno de ejecución:** El cargador prepara la infraestructura mínima para el Kernel, lo que incluye configurar el espacio de pila (stack space), habilitar el Modo Protegido para el caso de la BIOS (o Modo Largo en 64 bits) y preparar las variables de entorno recuperadas de la NVRAM.

Carga del sistema operativo

Una vez que el núcleo o Kernel ha sido cargado en la memoria RAM y recibe el control, el sistema operativo (SO) deja de depender de las funciones simplificadas del firmware para convertirse en el gestor absoluto de los recursos.

En sistemas modernos, el punto de partida es la llamada a la función **ExitBootServices()**. Esta llamada, realizada por el cargador del SO o el Kernel mismo, notifica al firmware que el sistema operativo está listo para tomar posesión del hardware.

Al ser notificado, el firmware libera todos los controladores y servicios que usó para iniciar (como el sistema de archivos FAT32 básico), permitiendo que el SO reclame esa memoria RAM para su propio uso. Solo los Runtime Services (como la gestión del reloj de tiempo real o el acceso a variables en NVRAM) permanecen activos para que el Kernel pueda consultarlos durante su funcionamiento normal.

Para operar con la eficiencia de un procesador moderno, el Kernel debe dejar de usar el direccionamiento físico plano (donde el código accede directamente a las direcciones de los chips de RAM).

- **Transición a Modo Virtual:** El Kernel suele invocar la función SetVirtualAddressMap() para convertir los servicios del firmware de un modo de direccionamiento físico a uno virtual. Esto permite que el SO gestione la memoria mediante paginación, protegiendo procesos entre sí.
- **Consumo del Mapa de Memoria:** El Kernel utiliza el mapa de recursos físicos entregado por el cargador (a través de la System Table) para saber exactamente qué áreas de la RAM están libres y cuáles están reservadas para el hardware.

Aunque el firmware ya inicializó el hardware básico, el sistema operativo realiza su propio proceso de descubrimiento exhaustivo para optimizar el rendimiento.

A diferencia de la BIOS tradicional, que intentaba inicializar todo el hardware antes del arranque, los sistemas modernos siguen una política de "activación perezosa" (lazy activation) bajo el Modelo de Controladores de UEFI, el firmware solo activa el conjunto mínimo de dispositivos necesarios para alcanzar el objetivo de arranque.

Esto delega al sistema operativo la responsabilidad de realizar un descubrimiento exhaustivo una vez que toma el control. Posteriormente se localizan y procesan las tablas críticas como ACPI (para gestión avanzada de energía y configuración) y SMBIOS (para información del sistema), que sirven como el "inventario" maestro del hardware disponible.

Este redescubrimiento del Hardware permite que el SO oculte los detalles complejos del chipset y presente al programador abstracciones consistentes como archivos o flujos de datos.

Con el hardware identificado, el Kernel sustituye los controladores genéricos del firmware por controladores específicos del fabricante que permiten explotar todas las capacidades del dispositivo.

En sistemas clásicos, esto implicaba leer archivos como CONFIG.SYS para cargar controladores de dispositivo y programas listados, posteriormente se activan las funciones de infraestructura esenciales: protocolos de red, subsistemas de sonido y controladores de video. Estos procesos operan a menudo como servicios persistentes o "daemons".

La **fase final** consiste en levantar la infraestructura que el usuario final realmente percibe. Se inicializa el subsistema de gestión de infraestructura, encargado de mantener el sistema funcional (como systemd o Services.exe), los cuales monitorean que los procesos y servicios requeridos por el usuario estén activos y respondan ante fallos.

A su vez, el Kernel lanza finalmente el intérprete de comandos o el entorno de escritorio (como el shell de Windows o el escritorio de Linux), presentando la interfaz de usuario (UI) definitiva y dejando la computadora lista para la interacción humana.

A partir de este momento, el sistema entra en su fase de Runtime (RT), donde el Kernel permanece siempre en ejecución, gestionando interrupciones de hardware y solicitudes de servicios de los programas mediante llamadas al sistema.

Conclusión

El arranque de una computadora comprende toda una secuencia de eventos técnicos y precisos que a simple vista, como usuarios, nunca apreciamos ya que es un proceso sumamente discreto y veloz.

El manejo de métodos de seguridad, desde el primer instante en el que encendemos nuestros dispositivos, nos muestra cómo una computadora está profundamente protegida ante múltiples situaciones. Si bien es una herramienta

sumamente poderosa y compleja, también es extremadamente delicada, por lo que con el paso del tiempo se han perfeccionado los distintos mecanismos tanto de protección como de inicialización. Esto ha permitido la transición desde sistemas tradicionales como la BIOS hacia soluciones más modernas como UEFI, las cuales ofrecen mayor flexibilidad, seguridad y capacidad de adaptación frente a las necesidades actuales, como el manejo de grandes volúmenes de datos y el uso intensivo de recursos.

Los métodos de arranque se encuentran en constante evolución, ya que no solo los sistemas operativos cambian con el tiempo, sino también las arquitecturas de hardware que los soportan. Por ello, aunque hoy en día es posible describir el proceso de arranque mediante etapas bien definidas: desde la estabilización de la energía, la liberación del procesador, la ejecución del firmware, hasta la carga del sistema operativo, es importante reconocer que estas etapas pueden seguir transformándose conforme surjan nuevas tecnologías.

Por lo tanto, el arranque de una computadora no debe entenderse como un proceso inmutable, sino como un sistema dinámico que se adapta continuamente a los avances tecnológicos. Esto implica que conceptos actuales como firmware avanzado, mecanismos de verificación de integridad o la gestión de recursos podrían seguir evolucionando, dando lugar a nuevas formas de interacción entre el hardware y el software desde los primeros instantes de funcionamiento del sistema.

Finalmente, comprender este proceso no solo permite apreciar la complejidad interna de una computadora, sino también reconocer la importancia de cada una de sus etapas, ya que cualquier fallo en ellas puede afectar al funcionamiento completo del sistema.

Bibliografía

- Fisher, T. (2023, 14 de junio). *What is BIOS (Basic Input/Output System)?* Lifewire. https://www.lifewire.com/bios-basic-input-output-system-2625820
- Gerousis, A. (2023, 22 de marzo). *How we test power supplies*. Tom's Hardware. https://www.tomshardware.com/reviews/how-we-test-psu,4042.html
- Hoffman, C. (2022, 12 de julio). *BIOS vs. CMOS: What's the difference?* How-To Geek. https://www.howtogeek.com/805136/what-is-cmos/
- Microsoft. (2023, 23 de mayo). *Configure MBR-based hard drive partitions*. Microsoft Learn.

https://learn.microsoft.com/en-us/windows-hardware/manufacture/desktop/configure-biosmbr-based-hard-drive-partitions?view=windows-11
- Microsoft Corporation. (2023). *How drive partitioning works: Master Boot Record and GUID partition table*. Microsoft Learn. https://learn.microsoft.com/
- Wikipedia contributors. (2025, September 9). *Advanced format*. Wikipedia. https://en.wikipedia.org/wiki/Advanced_Format
- Mueller, S. (1988). *Upgrading and repairing PCs*. Que Corporation.
- Zimmer, V., Rothman, M. y Marisetty, S. (2010). *Beyond BIOS: Developing with the Unified Extensible Firmware Interface* (2.ª ed.). Intel Press.